

Optimizing Large Language Models for Efficient Edge Deployment

Deepshika Poojary,^{a*} Saachi Suvarna^{a†}

^aDepartment of AIML, NMAM Institute of Technology, Nitte, 574110, India

Abstract

Over the years, LLMs have proven to be remarkably capable across a range of natural language processing (NLP) tasks, yet deploying these resource-intensive models on edge devices remains challenging due to limited computational capacity. This research explores a multi-technique optimization approach designed for edge deployment. We develop a custom GPT model, based on the GPT-2 architecture and trained on Shakespearean texts, and optimize it through iterative pruning and knowledge distillation. By selectively removing neurons, attention heads, and embedding dimensions with minimal impact on performance, we achieve a compression ratio of 35.96%. Knowledge distillation further compensates for performance loss due to pruning, reducing evaluation loss from 3.1399 to 2.0749. To experiment with efficient deployment, we utilize quantization on DistilGPT-2, compressing it to 4-bit precision, which reduces the model's disk size from 328 MB to 85 MB. Deployed through a Streamlit web application, the quantized model achieves efficient CPU-based inference, generating approximately 500 tokens in an average of 5.01 seconds. This study demonstrates the potential of deploying optimized LLMs on the edge, broadening the scope for real-time NLP applications in resource-limited environments.

Keywords: Large Language Models; LLM Compression; Generative AI; Edge AI

1. Introduction

1.1. Background

The field of natural language processing (NLP) has seen rapid advancements with the rise of large language models based on transformer architecture. Models such as GPT, BERT, LLaMa, and their derivatives have achieved impressive results in a variety of language tasks, ranging from text generation and summarization to question answering and sentiment analysis. However, the high computational and storage requirements of these models restrict their applicability in edge computing environments, where resources are constrained. Given the increasing demand for real-time NLP applications on edge

devices—such as personal computers (PCs), mobile phones, and IoT devices—there is a pressing need for optimization strategies that can balance model performance with resource efficiency.

1.2. Objective

The primary goal of this research is to create an optimized framework for deploying large language models on edge devices. This study focuses on reducing model size and computational demands without sacrificing the model's ability to generate content. By applying methods such as pruning, knowledge distillation, quantization, and distributed training techniques, we aim to explore ways to compress and optimize a LLM for efficient inference on CPUs.

* Corresponding author. e-mail: deepshikapoojary@gmail.com.

† Corresponding author. e-mail: saachisuvarna2003@gmail.com.

1.3. Motivation

Deployment of LLMs on the edge opens up a range of possibilities in applications that require low latency and high privacy standards. By reducing the computational load of LLMs without compromising on their content, our work contributes to making advanced NLP applications more accessible across various devices and use cases. Furthermore, this project aligns with current industry and research trends toward creating more sustainable and efficient AI models that can operate effectively within resource constraints.

1.4. Contributions

This research contributes to the field by developing an optimized LLM that balances performance and efficiency for edge device deployment. Key contributions include:

1. **Model Compression through Pruning:** We achieved a 35.96% compression rate by systematically removing neurons, attention heads, and embedding dimensions, thereby reducing computational demands.
2. **Knowledge Distillation for Performance Preservation:** Knowledge distillation mitigates the performance loss typically associated with pruning, improving evaluation loss from 3.1399 to 2.0749.
3. **Quantization for Efficient Inference:** By quantizing the model to 4-bit precision, we reduced the model's disk size from 328 MB to 85 MB, enabling efficient CPU-based inference.
4. **Real-Time Edge Deployment:** Through a Streamlit-based deployment, our model generates approximately 500 tokens in just over 5 seconds on a CPU, showing the potential of high-performance LLMs in constrained environments.

2. Related Work

The deployment of LLMs on edge devices has received increasing research attention, driven by the growing need for real-time, efficient applications outside of traditional cloud infrastructures. Edge

deployment of LLMs is challenging due to limitations in computational power, memory, and latency requirements. Recent literature highlights a range of model optimization techniques that reduce the complexity and size of LLMs while preserving their performance. This section discusses key contributions to model compression and edge deployment in the context of large language models.

2.1. Model Compression and Pruning Techniques

One of the primary techniques to reduce model size and improve efficiency is pruning, where specific components of the model—such as neurons, attention heads, or layers—are selectively removed based on their importance. Muralidharan et al. [1] present a structured approach to iterative pruning and distillation for large models, demonstrating the effectiveness of removing low-impact components while retaining core functionalities. Other works, including studies by Zhu et al. [2], explore both structured and unstructured pruning techniques, emphasizing the importance of balancing compression rates with minimal degradation in accuracy.

This project adopts a structured pruning approach similar to these studies, using importance scoring to prune neurons, attention heads, and embeddings in a custom GPT model.

2.2. Knowledge Distillation

Knowledge distillation is another widely researched method for improving the efficiency of LLMs, where a larger, fully trained "teacher" model transfers knowledge to a smaller "student" model, helping it approximate the teacher's capabilities with fewer parameters. Muralidharan et al. [1] also incorporate knowledge distillation into their iterative pruning strategy to mitigate the negative effects of compression on model accuracy.

This project uses a similar strategy, distilling knowledge from the unpruned custom GPT model to improve the performance of the pruned, compact version.

2.3. Quantization

Quantization reduces the precision of model parameters, significantly decreasing memory usage and computational overhead, which is particularly beneficial for inference on CPUs. Wang et al. (2024) [3] extensively reviewed quantization techniques and their impact on inference speed and memory usage for large models. In related work, Lee et al. (2024) [4] conducted a comparative study on the performance of different compression and quantization techniques for efficient LLM deployment on edge devices. Their findings indicated that the GGUF quantization scheme optimized CPU performance, while GPTQ [5] and AWQ [6] offered improved speed and efficiency on specific hardware setups. This study helped us choose the appropriate quantization technique for deploying large models on edge devices.

This project applies a 4-bit quantization technique on DistilGPT-2, transforming it into a highly compact model suitable for CPU inference on edge devices.

2.4. Parallelism for LLMs

Parallelism is an essential technique in large language model (LLM) training and deployment, especially as model sizes grow and demand higher computational resources. Various forms of parallelism—including model, data, tensor, and pipeline parallelism—have been explored to manage the high memory and processing requirements of LLMs.

Deepak Narayanan et al. (2021) [7] demonstrated significant efficiency gains by implementing model and data parallelism on GPU clusters, achieving a 10% increase in throughput for large models.

Megatron-LM [7] employs a combination of tensor and pipeline parallelism, distributing both model layers and tensor computations across multiple devices to maximize GPU utilization and manage memory more effectively. This hybrid approach is particularly advantageous for fine-tuning large models, as it provides flexibility and scalability across diverse hardware configurations.

GPipe, developed by Huang et al. [8], introduced a scheduling approach that pipelines the mini-batches across model layers distributed over multiple GPUs,

achieving near-linear speedup when scaling model training across devices.

In this project, pipeline parallelism and data batching were implemented during fine-tuning of the DistilGPT-2 model. This approach minimized training time by distributing computations across multiple GPUs, reducing idle time, and improving throughput.

2.5. Cloud-Edge Collaboration for Deployment of LLMs

The concept of distributed LLMs has gained traction as a strategy for efficient edge deployment. EdgeShard [9], proposed by Zhang et al., improves LLM inference efficiency on edge devices through collaborative edge computing, effectively reducing latency and bandwidth requirements compared to cloud-based solutions. PrivateLoRA [10] by Wang et al. addresses privacy and data locality concerns by distributing privacy-sensitive computations to edge devices while offloading shared computations to the cloud. This hybrid approach enables higher throughput compared to device-only solutions, achieving improved performance while addressing privacy needs.

By building on these existing methods, our project combines structured pruning, knowledge distillation, and quantization in a unique manner tailored for efficient edge deployment, aiming to reduce resource consumption while preserving the performance of the models.

3. Methodology

This study applies a multi-technique approach to optimize two models, a custom GPT model (based on the GPT-2 architecture) [11] trained on Shakespearean text and a pre-trained DistilGPT-2 model. Each model serves as a testing ground for specific optimization techniques aimed at enabling efficient edge deployment while minimizing degradation in language generation quality. This section details the model development, pruning and distillation process for the custom GPT model, and model parallelism and quantization for the DistilGPT-2 model.

3.1. Pruning Techniques

To reduce the custom GPT model's computational complexity, we implemented structured pruning by assessing the importance of neurons, attention heads, and embedding dimensions. This process involved the following steps:

1. **Neuron Pruning:** We calculate the importance scores for each neuron by aggregating the activations across batch and sequence dimensions. Neurons with low activation scores are pruned, which leads to smaller linear layers in each transformer block. High-importance neuron weights are retained to minimize loss of functionality.
2. **Attention Head Pruning:** In the multi-head attention mechanism, we compare the L2 norm of each attention head's output computed to rank their significance. This way, low-importance attention heads are pruned, while weights for the most critical heads are preserved to maintain effective attention.
3. **Embedding Pruning:** We evaluate the importance of each embedding dimension to implement the selective pruning of less important dimensions. The resulting model maintains only the most significant embeddings, contributing to overall model efficiency.

The increase in model's evaluation loss post-pruning is subsequently addressed through knowledge distillation.

3.2. Knowledge Distillation

To mitigate the performance degradation from pruning, we use knowledge distillation. In this approach, the original, unpruned custom GPT model served as a "teacher," providing guidance to the pruned "student" model through its output logits. During training, the student model was optimized to replicate the teacher model's responses, learning to approximate its capabilities with a smaller parameter set. Knowledge distillation helped the pruned model recover much of its initial performance.

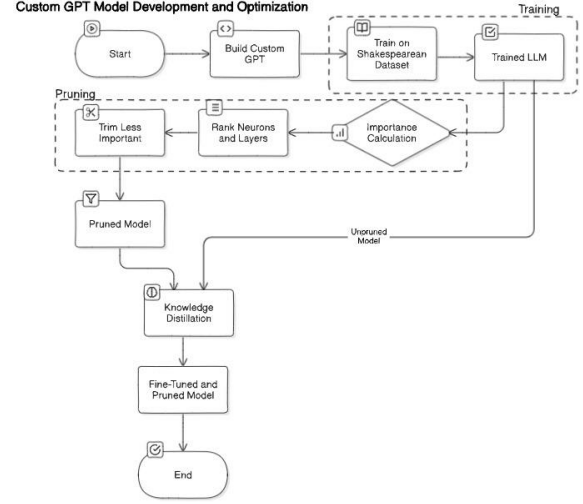


Fig. 1. Model Pruning and Knowledge Distillation on custom GPT

3.3. Model Parallelism for Distributed Training

To expedite training and maximize computational efficiency, we implemented model parallelism by distributing the layers of DistilGPT-2 across two GPUs. This parallelization approach implements vertical model parallelism, enabling different sections of the model to run concurrently on separate GPUs, reducing overall training time. We use pipeline parallelism to minimize idle time during sequential layer processing and, as a result, reduce training time.

3.4. Quantization

Quantization was applied to DistilGPT-2 to explore efficient inference on CPU for edge deployment. The quantization process involved converting the model's weights and activations from 16-bit floating-point precision to 4-bit integers using Q4_K_M quantization. This conversion would decrease the model's size to almost a quarter of the original size, considerably reducing memory requirements and improving inference speed.

After quantization, the model was transformed into the Generic GPT Unified Format (GGUF) using Llama.cpp, making it compatible with CPU architectures. This format supports efficient CPU-based inference, ideal for deployment on devices without GPUs.

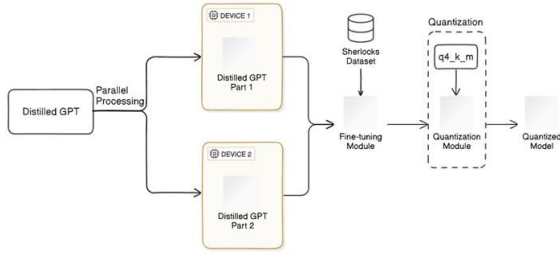


Fig. 2. Model Parallelism and Quantization on DistilGPT-2

4. Results

This section presents the results of the pruning, knowledge distillation, and quantization techniques applied to the custom GPT and DistilGPT-2 models, respectively, and evaluates their performance in terms of model compression, inference speed, and deployment efficiency.

4.1. Model Compression using Pruning and Knowledge Distillation

The custom GPT model, trained on Shakespearean texts, underwent iterative pruning and distillation, achieving significant compression while retaining performance. Table 1 presents a summary of model size and loss metrics before and after pruning and distillation.

Pruning reduced the model's parameter count by approximately 36%, significantly reducing its size without major performance degradation. The initial increase in evaluation loss following pruning (from 1.4788 to 3.1399) was largely addressed through knowledge distillation, lowering it to 2.0749.

Table 1

Summary of key metrics before and after optimization

Metric	Pre-Pruning	Post-Pruning	Post-Distillation
No. of Parameters	3.22 M	2.06 M	2.06 M
Compression Ratio	-	35.96%	35.96%
Evaluation Loss	1.4788	3.1399	2.0749

4.2. Quantization and Inference Performance on Edge Devices

The DistilGPT-2 model, quantized to 4-bit precision (FP16 to INT4), was tested for inference on a CPU to simulate real-world edge deployment scenarios. Despite the reduction in disk size to 85 MB, the quantized model maintained its capability to generate 500 tokens in an average of 5.01 seconds across several inference trials on a CPU. This performance aligns with real-time application requirements, confirming that the quantized model can deliver efficient inference on standard edge devices without GPU acceleration.

4.3. Comparison of Parallelization Approaches

In addition to pruning and quantization, parallelization methods were implemented to expedite the fine-tuning of DistilGPT-2. The model was fine-tuned for 10 epochs using different parallelization setups.

Figure 3 shows the difference in execution times for various parallelization setups, whereas Figure 4 helps us visualize the speedup achieved by the Model Parallelism and Pipeline Parallelism approach, relative to a single GPU.

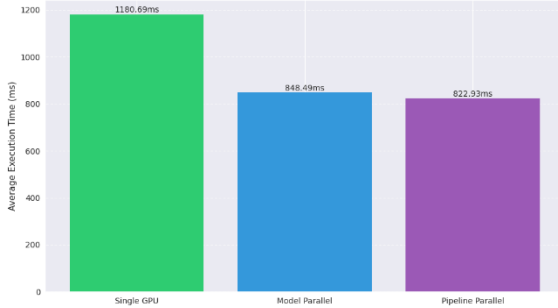


Fig. 3. Comparison of Different Parallelization Approaches

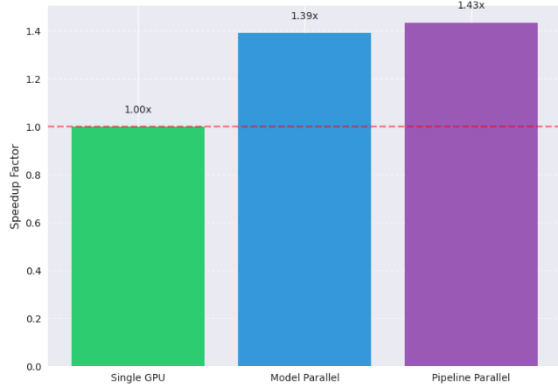


Fig. 4. Speedup Relative to Single GPU

Pipeline parallelism achieved the lowest execution time, highlighting its effectiveness in reducing idle times and improving GPU utilization. This result indicates that pipeline parallelism offers an effective solution for scaling LLM training on multi-GPU systems.

4.4. Deployment

The optimized models were deployed using Streamlit, a lightweight web framework that enabled real-time interaction with the models.

5. Conclusion

This project explores various approaches for optimizing LLMs to enable efficient deployment on edge devices. By combining structured pruning,

knowledge distillation, quantization, and parallelized fine-tuning, we showed that substantial model compression and computational efficiency can be achieved without significant loss in performance. The integration of these techniques not only reduces the model's memory and processing requirements but also facilitates real-time inference on standard CPUs.

Through this work, we highlight the potential of optimized LLMs to operate beyond traditional high-resource infrastructures, expanding their applicability to diverse real-world applications, particularly in settings where computational resources are limited.

6. Future Work

Further enhancements can be explored to improve model efficiency, scalability, and adaptability:

- **Advanced Quantization Techniques:** Future work could explore more aggressive quantization methods, such as low-bit (1-bit) [12][13] or ternary quantization [14], to further reduce the model's memory footprint and enable deployment on smaller, resource-limited devices such as mobile platforms.
- **Advance Parallelization Techniques:** Expanding beyond model parallelism, incorporating data, tensor [15], sequence or 3D [16] parallelism could allow for even larger models or datasets to be handled efficiently across multiple GPUs. Such methods would improve scalability and considerably improve training efficiency.
- **Further Pruning and Compression:** Additional research into advanced techniques, such as one-shot pruning, lottery ticket hypothesis-based pruning [17] and dynamic sparsification could yield further model size reductions and better inference speed, enabling deployment on an even wider range of devices.
- **Real-Time Monitoring and Adaptive Optimization:** Implementing real-time performance monitoring and adaptive optimization techniques could allow the model to adjust its configuration dynamically, optimizing performance based on current workload and resource availability. This would be particularly useful for deployment in environments with fluctuating demands or limited power availability.

Acknowledgments

The success of this project would not have been possible without the effort, assistance, and guidance of many individuals. We would like to express our sincere gratitude to everyone who contributed to the completion of this research.

We are deeply thankful to NMAM Institute of Technology for providing us with the opportunity to pursue and fulfill our academic goals.

Our heartfelt thanks go to Dr. Niranjan Chiplunkar, Principal of NMAMIT, Nitte, for granting us the opportunity to undertake this project and for providing the necessary resources and support throughout.

We would also like to acknowledge the invaluable guidance of Dr. Sharada U Shenoy, Head of the Department of Artificial Intelligence and Machine Learning, NMAMIT. Her support, expertise, and insightful feedback were instrumental in shaping the success of this project.

Additionally, we extend our thanks to all the faculty members and non-teaching staff of the Department of Artificial Intelligence and Machine Learning for their continuous support and encouragement.

References

- [1] S. Muralidharan, S. T. Sreenivas, R. Joshi, M. Chochowski, M. Patwary, M. Shoenybi, M. Catanzaro, J. Kautz, and P. Molchanov. "Compact language models via pruning and knowledge distillation." arXiv preprint arXiv:2407.14679, 2024.
- [2] X. Zhu, J. Li, Y. Liu, C. Ma, W. Wan, "A survey on model compression for large language models." arXiv preprint arXiv:2308.07633, 2023
- [3] W. Wang, W. Chen, Y. Luo, Y. Long, Z. Lin, L. Zhang, B. Lin, D. Cai, X. He, "Model compression and efficient inference for large language models: A survey." arXiv preprint arXiv:2402.09748, 2024.
- [4] A. Chavan, R. Magazine, S. Kushwaha, M. Debbah, and D. Gupta, "Faster and Lighter LLMs: A Survey on Current Challenges and Way Forward." arXiv preprint arXiv:2402.01799, 2024.
- [5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers." arXiv preprint arXiv: 2210.17323, 2023.
- [6] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration." arXiv preprint arXiv: 2306.00978, 2023.
- [7] D. Narayanan, M. Shoenybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, and A. Phanishayee, "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM." SC21: International Conference for High Performance Computing, Networking, Storage and Analysis (2021): 1-14.
- [8] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, "Gpipe: Efficient training of giant neural networks using pipeline parallelism." Advances in neural information processing systems, 32, 2019.
- [9] M. Zhang, J. Cao, X. Shen, Z. Cui, "EdgeShard: Efficient LLM Inference via Collaborative Edge Computing." arXiv preprint arXiv:2405.14371, 2024.
- [10] Y. Wang, Y. Lin, X. Zeng, and G. Zhang, "PrivateLoRA For Efficient Privacy Preserving LLM." CoRR, vol. abs/2311.14030, 2023.
- [11] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners." OpenAI blog, 1(8), 9, 2019.
- [12] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, "Bitnet: Scaling 1-bit transformers for large language models." arXiv preprint arXiv:2310.11453, 2023.
- [13] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, "The era of 1-bit llms: All large language models are in 1.58 bits." arXiv preprint arXiv:2402.17764, 2024.
- [14] C. Zhu, S. Han, H. Mao, and W.J. Dally, "Trained ternary quantization." arXiv preprint arXiv:1612.01064, 2016.
- [15] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using gpu model parallelism." arXiv preprint arXiv:1909.08053, 2019.
- [16] D. Team and R. Majumder, "DeepSpeed: Extreme-scale model training for everyone." <https://www.microsoft.com/en-us/research/blog/deepspeedextreme-scale-model-training-for-everyone/>, 2020.
- [17] J. Frankle, and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks." arXiv preprint arXiv:1803.03635, 2018